HoopaDex — Architecture Review

Date 2026-08-03 · **Reviewed at** v5.8 · **Findings shipped as** v5.9 · **Fixes shipped as** v5.10 **Repository** `willhoop/hoopadex` · **Live** https://willhoop.github.io/hoopadex/app/

Every number in this document came from running something. Where a figure could not be reproduced, it is marked as such rather than repeated. Where a fix could not be verified, it was left undone and recorded in section 6.

---

1. Executive summary

HoopaDex is a generation-accurate Pokédex that ships as a web page. Its stated value is that the numbers are right for the generation you are looking at, and its stated method is that any value derivable from a published source must be derived rather than typed. That method works: where it has been applied, the data is correct.

The problem is that the test suite could not tell the difference between the data being correct and the data merely being present.

I established this by mutation testing — breaking the shipped code on purpose, one bug at a time, and running the whole battery against each broken copy. **Five of ten deliberate bugs passed all 23 suites undetected.** Three of the five were in the damage calculator, which is the most numeric thing the app shows a reader and which had no test that computed a damage number at all. A fourth changed a Pokémon's historical base stats; a fifth changed which Pokémon are legal in a tournament format.

The suites were not bad. 778 assertions is a serious investment and much of it is genuinely load- bearing — the Generation I Special stat, the item introduction table and the regulation item diff all have committed derivations with drift checks, and all three caught their mutation immediately. The gap was structural and predictable: **tables with a derivation were defended; tables without one were defended only by whichever handful of entries someone had thought to pin.** PAST_STATS had 58 species and 8 of them pinned.

What was fixed

All eleven mutations are now caught, including one added to cover the stale-copy problem below.

- The damage arithmetic was extracted into pure functions and given 43 hand-computed assertions.
- A generation-blind critical-hit multiplier was corrected. Crits were ×2 through Generation V and ×1.5 from VI; the app used 1.5 everywhere, so every pre-VI critical hit read 25% low.
- `PAST_STATS` now has a derivation. `build/generate-past-stats.js` rebuilds all 58 species from Showdown's per-generation mods. **The shipped table reproduced exactly — 58 of 58, zero disagreements.** The data was right; only the proof was missing.
- `data/mega-abilities.json` and `data/regulations.json` gained drift checks. Six of nine committed data files are now checked, against three before.

- A **version 1.92 copy of the entire app was live** at `/app/HoopaDex_1_92.html`, returning HTTP 200. It predated every data fix this project has made. It is removed, and a test now fails if any page other than `index.html` appears in `app/`.

Everything below was written at v5.9, when the findings were recorded but most were still open. The fixes landed in **v5.10** and each finding carries its own status. The headline change is that **the unattended publisher is gone** — process killed, Startup shortcut removed, and both the script and its installer now refuse to run. HoopaDex publishes through `build/publish.sh`, which runs the suites and refuses to push on red.

The mutation set is committed as `build/mutation-check.js` and runs in CI, so the central finding of this review cannot quietly stop being true.

What is still open

- **The Bulk tab's objective is a modelling choice, not a derivation**, and the alternative model picks a different answer for 135 of 208 Pokémon. The claim is corrected and the tab now states its model, but **the recommendation is unchanged** — which model to serve is a product decision. Finding **F4**.
- `@smogon/calc` **still has no known version.** It is now pinned by SHA-256 so it cannot change unnoticed, but a checksum pins the artefact, not its provenance. Finding **F8**.
- **Two published statistics were withdrawn as unreproducible.** Section 4.
- **Nothing tests what the app looks like.** Screenshots would not composite in this environment; every visual claim here rests on reading the DOM. Section 6.

A correction to this review

Finding **F10** originally reported the white paper and deck as badly stale. That was wrong — it measured each document's own revision number instead of the app version it describes, and both were current. The false finding, why it happened, and the check that now measures the right number are in F10. It is the same error as the withdrawn statistics in section 4, from the other direction: a figure that is easy to grep for is not the figure you wanted.

What I could not verify

Screenshots would not composite in this environment, so every visual claim here rests on reading computed values out of the DOM, not on looking at the page. The items the previous handoff listed as "measured, never seen" remain unseen. Section 6 is the full list.

---

2. Findings, ranked by blast radius

Ranking is by what could put a wrong number in front of a reader, not by how untidy it is.

F1 — An unattended timer publishes to the live site every ten minutes, with no test gate

Severity: highest. This is the mechanism by which every other defect reaches a reader.

What is wrong. `C:\Users\willj\Projects\auto-publish.bat`, started by `START-AUTO-PUBLISH.bat` via a hidden VBScript in the Windows Startup folder, loops forever over six repositories running:

```
git add -A
git commit -q -m "auto: %date% %time%"
git push -q origin main
```

There is no test run anywhere in it — `grep -ci test auto-publish.bat` returns `0`. Whatever is in the working tree at the ten-minute mark becomes the live public site.

How I proved it. During this review the timer committed my in-progress work twice, unasked:

```
$ git log --oneline -3
6d11275 auto: Mon 08/03/2026 17:41:50.81
7666f41 auto: Mon 08/03/2026 17:31:30.52
8c24b3f Add docs/HANDOFF.md
```

`git show --stat 6d11275` lists eight files including `app/index.html` and the deletion of `app/HoopaDex_1_92.html`. Both commits are authored `willhoop <willjhooper@msn.com>`, so the history cannot distinguish a human decision from a timer firing.

What it could put in front of a reader. Any intermediate state. The previous handoff records a session in which a shell heredoc left the app a syntax error — a blank page — with all 18 suites still green; that blank page was published by this mechanism. I checked whether the two commits it made during this review were internally consistent:

commit	`app/index.html`	CHANGELOG top	consistent?
7666f41	5.8	5.8	yes
6d11275	5.9	5.9	yes

Both happened to land on clean boundaries. That is luck. The timer has no knowledge of whether an edit is finished.

Status: FIXED in 5.10 — the script is dead.

The script existed for a reason I did not know when I wrote the paragraph above: Claude Cowork would not push to GitHub itself, so for some sessions the timer was the *only* route to the live site. That is a real problem and a timer is a reasonable answer to it, so my first fix kept the timer and gated it. Will then confirmed he is not using Cowork again, which removes the only argument for an unattended publisher.

It is now retired in four places so it cannot return: the running process killed, the `CHOMP-autopublish.lnk` Startup shortcut removed, and both `auto-publish.bat` and its installer `START-AUTO-PUBLISH.bat` exit

immediately with an explanation of why and what to use instead. The original script body is preserved beneath the new header, because its comments record what two publishers on one repository cost.

The gate built for that intermediate fix survives as `Projects\publish-gate.js` and remains useful on its own — `node publish-gate.js <repo-dir>` reports whether a repository is safe to publish. It runs the repository's suites **and** separately checks that `app/index.html` parses — separately, because a green suite does not imply a page that runs: the suites slice text out of the file, and text slices perfectly well when it cannot execute. That is exactly how the blank page shipped. On failure it writes the reason to `autopublish.log` and skips the push.

Verified two ways. It passes all five published repositories as they stand, so adding it stops nothing that works today:

```
portfolio                PASS (safe to publish)
Pokemon/HoopaDex         PASS (safe to publish)
Pokemon/CHOMP            PASS (safe to publish)
Pokemon/KaizoDex         PASS (safe to publish)
jeopardy-wagering        PASS (safe to publish)
```

And it blocks a copy of this app with one closing brace removed — the blank-page failure — with the batch block tested in real `cmd`, not merely reasoned about:

```
RESULT=BLOCKED
[Mon 08/03/2026 18:07:55.87] testrepo: NOT PUBLISHED - app/index.html does not parse:
  Unexpected token ')' | suite failed: tests/test-syntax.js
```

A repository with no `tests/` and no `app/index.html` is published exactly as before, so this cannot strand a project that never had tests.

`build/publish.sh` is added for sessions that *can* push directly: parse check, suites, size guard, push, then verify the commit reached origin and that Pages actually served the new version. One repo, one publisher — with the timer as the gated fallback rather than an ungated competitor.

F2 — The damage calculator had no numeric test

Severity: high. It is the most numeric output in the app.

What is wrong. Nothing in the 23-suite battery ever computed a damage number. Coverage of the calculator consisted of regex assertions that named functions exist.

How I proved it. Three mutations, each run against all 23 suites:

```
M5  *** SILENT ***  critical hits multiply by 2.5x instead of 1.5x     red suites: NONE
M6  *** SILENT ***  STAB becomes 1.9x instead of 1.5x                  red suites: NONE
M7  *** SILENT ***  drop the 0.75 spread-move reduction                red suites: NONE
```

What it could put in front of a reader. Any damage figure, any KO probability. A reader building a team around "guaranteed 2HKO" would have had no way to know.

This is worse than "it is only the fallback engine". The app is published as a portable single file; `calc-engine.js` is a *separate* 480 KB sibling. Open `index.html` on its own — which the technical documentation instructs readers to do — and `SmogonCalc` is undefined, so this local code **is** the calculator. `calcRun()` falls back to it on any exception with only a `console.warn`, and the reader is never told which engine produced the number.

Status: FIXED. The arithmetic is now `calcLocalRolls`, `calcLocalStab` and `calcLocalCritMult`, pure functions outside the DOM handler, with `tests/test-damage-formula.js` — 43 assertions whose expected values are worked by hand from the damage formula in the comments rather than read back out of the app. Verified: all three mutations now fail it, and the functions were exercised in the live page, returning the hand-computed rolls `39…46`.

An honest note on the process: my first attempt at this refactor moved where STAB is *applied* into the pure function but left where it is *derived* in the DOM handler, and M6 went straight back to passing. Moving a value somewhere testable is not the same as moving the decision that produces it. I only caught it because I re-ran the mutation instead of trusting the green suite.

F3 — Critical hits were generation-blind in a generation-accurate dex

Severity: high. A wrong number, shipped, in the app's core promise.

What is wrong. `const critM=crit?1.5:1;` — no generation term. A critical hit was a ×2 multiplier from Generation II through V and became ×1.5 in Generation VI.

How I proved it. Read from source at `app/index.html`, then confirmed against the shipped behaviour after extraction. `CLAUDE.md` states: "*If a change would make the app show current- generation data for an older generation, it is a bug, not a simplification.*" This qualified.

What it could put in front of a reader. Every critical hit in Generations II–V read 25% low.

Status: FIXED. `calcLocalCritMult(gen)` returns 2 below Generation VI and 1.5 from VI. Verified live in the page: the same crit returns 92 in Gen V and 69 in Gen IX.

F4 — The Bulk tab's objective is a modelling choice presented as a derivation

Severity: high, because it is a claim about mathematics that other documents repeat.

What is wrong. `docs/BACKLOG.md`, `CHANGELOG.md` and `docs/HANDOFF.md` all stated:

> Derived: total hits survived scales with HP × (Def + SpD)

That is true under one model and false under another, and it was stated as though it were the only mathematics. Both of the following are correct:

Model of the opponent	Maximise	Shipped
Fully physical **or** fully special, even chance of each — *expected* hits survived	HP × (Def + SpD)	yes
A mixture of both **within one battle** — hits actually survived	HP × Def × SpD / (Def + SpD)	no

The first averages over two separate battles; the second is one battle. HP × (Def + SpD) is dominated by the *larger* defence, so it over-rewards HP, while real survivability is bottlenecked by the *weaker* one.

How I proved it. I ran the shipped `optimalBulkSpread` against all 208 Pokémon of Regulation M-B with real base stats from PokéAPI, at 32 SP, cap 32, neutral nature, and compared it to an exhaustive search under the mixed-damage objective:

```
spreads where shipped != hits-survived optimum : 135 of 208 (64.9%)
mean shortfall in hits survived, those cases   : 0.95%
worst case: avalugg (95/184/46) shipped 32/0/0, better 0/0/32, 10.49% fewer hits survived
```

Checked by hand for Avalugg: shipped gives HP 202, Def 204, SpD 66 → 202 ÷ (0.5/204 + 0.5/66) = 20,144. The alternative gives HP 170, Def 204, SpD 98 → 22,508. That is 10.5% more. The shipped answer does correctly maximise its *own* objective (54,540 against 51,340), which is the point: the arithmetic is right and the question was never stated.

The app's own Bulk intro additionally still asserted that a point of HP "is worth about twice what a single defence point is" — the folk rule the tool was built to replace.

What it could put in front of a reader. Defensive spreads that are up to 10.5% worse than optimal for the reader's actual situation, presented as "the exact optimum, searched rather than guessed".

Status: PARTIALLY FIXED. The claim is corrected in `docs/BACKLOG.md` and `CHANGELOG.md`, and the Bulk tab now states which question it answers and drops the folk-rule sentence — verified by reading the rendered paragraph back out of the live DOM. **The recommendation itself is unchanged**: which model to serve is a product decision about how readers actually play, not a defect I should decide alone. That call is Will's, and the numbers to make it are above.

F5 — A version 1.92 copy of the whole app was published and reachable

Severity: high. A complete, authoritative-looking, wrong dex with a live URL.

How I proved it.

```
$ curl -s -o /dev/null -w "%{http_code} %{size_download}\n" \
    https://willhoop.github.io/hoopadex/app/HoopaDex_1_92.html
200 448760
```

Its second line reads `HOOPADEX VERSION: 1.92`. Measured against the current file, its `PAST_STATS` covers 44 species where today's covers 58, it has no entry for Krookodile (id 553) — the exact defect recorded as fixed in 1.96 — and its Generation I type chart predates the 1.97 correction.

What it could put in front of a reader. Every data error this project has spent versions correcting, rendering exactly as convincingly as the corrected app.

Status: FIXED. Removed from the published tree; git history retains it. `tests/test-syntax.js` now fails if any page other than `index.html` appears in `app/`, verified by mutation (M11). Its documentation in `README.md` and `docs/HOOPADEX-technical-docs.md` — which told readers to open it — was corrected in the same pass.

F6 — Two derived tables were generated, committed, then never checked again

Severity: medium-high.

How I proved it.

```
M9   *** SILENT ***  CHAMPIONS_IDS_MA: drop Venusaur (3) from Reg M-A    red suites: NONE
M10  *** SILENT ***  CHAMP_MEGA_ABILITIES: Mega Barbaracle -> Levitate   red suites: NONE
```

The roster case is the more instructive. Every assertion in `test-champions-roster.js` was *relational* — it compared the rosters to each other. Because M-B is constructed as M-A plus additions, deleting an id shrank both sides together and `M-B == M-A + additions` still held. A uniformly wrong roster was structurally undetectable.

At the time, three of eight committed data files had a drift check.

What it could put in front of a reader. A wrong tournament legality list, or a wrong ability on a Mega — both of which look identical to right ones.

Status: FIXED. `tests/test-mega-abilities.js` compares every pokemon+ability pairing against the Showdown-derived file and is a genuine correctness check. The roster check is weaker by nature and labelled as such in the code: `data/regulations.json` is generated *from* the app, so it cannot prove the roster is correct — only that it cannot change without the committed artefact changing too. Six of nine data files are now checked. Both mutations now fail.

F7 — PAST_STATS had 58 species and 8 of them tested, with no derivation

Severity: medium-high, and the most improved by this review.

How I proved it.

```
M1  *** SILENT ***  PAST_STATS: wrong Gen IX Attack for Zacian    red suites: NONE
```

`test-past-stats.js` pinned ids 25, 26, 49, 51, 85, 488, 553 and 681. The table has 58 entries, so 50 species could be given any value at all. This is the table measured at 10 of 43 entries correct before version 1.96 — the project's own worst data failure, still without a derivation.

Status: FIXED, and the news is good. `build/generate-past-stats.js` rebuilds the table from Showdown's `data/mods/genN/pokedex.ts`, which record only what differs from the current game — so a `baseStats` line in the gen5 mod *is* the Generation V value. Result:

```
wrote data/past-stats.json: 58 species, {"gen5":29,"gen6":25,"gen7":1,"gen8":3}
app species: 58   derived species: 58
in app only    : 0
in derived only: 0
disagreements  : 0
```

The shipped table was already exactly right. What was missing was the proof, and it is now a drift check that fails on M1.

A note on method: my first run fetched only mods 6–8 and reported 29 species as unsourced. That was my error, not the app's — a stat that changed *at* Generation VI has its prior value in the **gen5** mod. Reporting those 29 as unsourced would have been a false accusation against correct data.

F8 — "One HTML file, no dependencies" is not true

Severity: medium. It misdescribes the artefact in every document.

How I proved it. `app/` contains `index.html` (645 KB), `calc-engine.js` (481 KB), `champions-learnsets.json` (1.4 MB) and, until this review, `HoopaDex_1_92.html`. `index.html` line 10 loads a stylesheet from `fonts.googleapis.com`, so the page also makes a third-party network request and is not fully portable offline.

`calc-engine.js` is a vendored build of `@smogon/calc`. **No version is recorded anywhere** — a grep across `CLAUDE.md`, `README.md`, `docs/` and `CHANGELOG.md` finds the package named seven times and its version zero times. There is no lockfile, no upstream commit, no checksum and no test.

What it could put in front of a reader. Indirectly: the primary damage engine cannot be audited, reproduced or safely upgraded, and a silent fallback to the local engine changes the answer because the local path ignores held items entirely (`let itemMult=1; // items handled by Smogon engine`).

Status: PARTIALLY FIXED. The technical documentation now records that the engine is vendored with no version. Pinning it properly means re-deriving the bundle from a known upstream commit, which I did not attempt — see section 6.

F9 — The item table's drift check cannot detect the universe growing

Severity: low today, structural. A good example of a check that passes on broken code.

What is wrong. `test-generation-tables.js` asserts `Object.keys(derived.gens).length === Object.keys(ITEMS).length` — 325 === 325. Both sides come from the same generated file, so the check confirms the app matches the derivation but cannot notice that the derivation is missing items. Unknown items then fall through `ITEM_INTRO_GEN[name] || 9`, silently becoming Generation IX.

How I proved it. I fetched the 18 held-item categories the app actually loads:

```
items the app loads      : 370
present in ITEM_INTRO_GEN: 325
MISSING -> silently Gen IX: 45 (12.2%)
  cat 44: 45 missing  e.g. clefablite, victreebelite, starminite, dragoninite
```

What it could put in front of a reader — and the honest answer. *Nothing, today.* All 45 are Legends: Z-A mega stones, which really are Generation IX, so the `|| 9` default is correct for every item currently affected. I checked this rather than assuming it, and it downgrades the finding from "live wrong data" to "unsound mechanism". The risk is the next item PokéAPI adds to an *older* category, which would be silently hidden from every generation before IX.

Status: NOT FIXED. Fixing it properly means regenerating the item table against the live PokéAPI item universe, and a network-dependent assertion does not belong in a suite that must pass offline. Recommended approach in section 7, rule **R4**.

F10 — Documentation drift, unchecked

Severity: low for readers of the app, high for anyone inheriting the project.

CORRECTED 5.10 — this finding was substantially wrong, and the error is instructive.

I first measured it by taking the highest version-like string in each document. That produced a table showing the white paper at 5.3 and the deck at 1.3 against an app at 5.9, and I reported the deck as roughly fifty versions stale.

It was not. `1.3` is the **deck's own revision number**, which is a different and perfectly legitimate number. Measuring the stamp that actually names the app:

Document	App version it claims to describe	App version at review
`docs/HOOPADEX-whitepaper.md`	HoopaDex v5.8	5.8 — **current**
`docs/HOOPADEX-deck-plain-english.md`	HoopaDex v5.8	5.8 — **current**
`docs/HOOPADEX-technical-docs.md`	HoopaDex v5.8	5.8 — **current**
`README.md`	no app version stamp at all	—

All three were current when this review began. They went stale only because *this review* bumped the version. I published a false accusation against correct documents, and it is the same error the project has met twice before: **a number that is easy to grep for is not the same as the number you want.** The bulk figures in section 4 failed the same way, from the other direction.

What survives of the finding is narrower and still true: the rule had no check, `README.md` carried no stamp to check, and the white paper's §5 did contain a genuinely false claim — that every suite had been mutation-checked, which section 2 disproves.

`CLAUDE.md` requires white paper, deck and technical documentation to be updated in the same pass as any change. `check_projects.py` verifies those files *exist* and that the CHANGELOG version matches the artefact — it does not and cannot verify that their contents are current. The rule therefore has no check, which by this review's own standard makes it a preference.

Worse, the gate is currently red for an unrelated reason:

```
$ python build/check_projects.py > /dev/null; echo $?
1
CHOMP          changelog=2.5.0  file=2.9  MISMATCH
```

HoopaDex passes; CHOMP fails; the script exits 1. Followed literally, `CLAUDE.md` forbids publishing HoopaDex until CHOMP is fixed. In practice that trains you to walk past the gate — which is why the version/CHANGELOG assertion has been duplicated into `tests/test-syntax.js`, where it must pass.

Status: FIXED in 5.10. `tests/test-doc-versions.js` requires the white paper, deck and technical documentation each to carry a `HoopaDex vX.Y` stamp equal to line 2 of the app, and to keep their own revision number distinguishable from it. The white paper's §5 has been rewritten and §5.2 added; the deck's slide 12 carried the same false claim and the same stale counts, and now carries neither. This does not prove a document's *contents* are current — nothing automated can — but the checkable part of the rule is now checked, so R8 and the documentation half of R11 move from preference to rule.

F11 — A handoff claim that was not true

Severity: low, but it wasted my time and would waste the next reader's.

`docs/HANDOFF.md` states: "*Every suite accepts a HOOPADEX_SRC env override so you can point it at a mutated copy.*" Measured, **nine of twenty-three did not**: `test-champions-roster`, `test-dex-search`, `test-hash-routing`, `test-move-tags`, `test-past-stats`, `test-past-types`, `test-paste-import`, `test-regulations`, `test-viz-palette`.

Status: PARTIALLY FIXED. The two suites this review needed (`test-champions-roster`, `test-past-stats`) now accept it; seven of twenty-five still do not. Mutation testing was done through a mirrored copy of the tree instead, which works for every suite.

---

3. Changes made

Test results before and after:

	Suites	Assertions	Mutations caught
Before	23	778	5 of 10
After	27	871	11 of 11, re-run in CI

All 25 suites pass. Command and output:

```
$ for f in tests/test-*.js; do node "$f"; done
suites: 27   failing: 0   total assertions: 871
```

File	Change	Why
`app/index.html`	Extracted `calcLocalRolls`, `calcLocalStab`, `calcLocalCritMult` from `calcRunLocal`	F2 — the arithmetic was unreachable from a test
`app/index.html`	Crit multiplier is now generation-aware	F3 — ×2 through Gen V, ×1.5 from VI
`app/index.html`	Bulk tab intro states its model; folk-rule sentence removed	F4
`app/index.html`	Version 5.8 → 5.9	Versioning rule
`app/HoopaDex_1_92.html`	**Deleted** (6,797 lines)	F5 — a stale wrong dex with a live URL
`tests/test-damage-formula.js`	**New**, 43 assertions	F2, F3
`tests/test-mega-abilities.js`	**New**, 10 assertions	F6
`tests/test-past-stats.js`	Added derivation drift check; accepts `HOOPADEX_SRC`	F7, F11
`tests/test-champions-roster.js`	Added anchor to `data/regulations.json`; accepts `HOOPADEX_SRC`	F6, F11
`tests/test-syntax.js`	Version/CHANGELOG agreement; no stray pages in `app/`	F5, F10
`build/generate-past-stats.js`	**New** generator	F7
`data/past-stats.json`	**New** derived data, 58 species	F7
`docs/BACKLOG.md`	Corrected the bulk claim; withdrew two figures	F4
`docs/HOOPADEX-technical-docs.md`	Removed 1.92 instructions; noted the unversioned engine	F5, F8
`README.md`	Removed the 1.92 "local snapshot" row	F5
`CHANGELOG.md`	5.9 entry	Versioning rule

---

4. Numbers corrected

Every location was checked, not just the first. `docs/HANDOFF.md` is left as a historical record of the previous session and is *not* rewritten; it is listed here so the discrepancy is visible.

Figure	Old value	Corrected value	Appears in	Status
HP/(2×Def) at the bulk optimum, mean	1.30	**0.919**	`docs/BACKLOG.md`, `CHANGELOG.md` (5.0 entry), `docs/HANDOFF.md`	Withdrawn as unreproducible; BACKLOG + CHANGELOG corrected. HANDOFF left as history.
HP/(2×Def), range	0.43 – 5.50	**0.41 – 1.42**	same three	same
HP/(Def+SpD) at the optimum, mean	0.90	**0.890**	same three	Reproduced. Now carries its provenance.
Critical-hit multiplier, Gens II–V	1.5	**2**	`app/index.html`	Fixed in code
Suites / assertions	23 / 778	**27 / 871**	`README.md`, `docs/HANDOFF.md`	Measured this review
`PAST_STATS` species covered	(untracked)	**58, all derived**	`data/past-stats.json`	New
Pre-1.96 `PAST_STATS` accuracy	"10 of 43 correct"	**7 of 42 agree with Showdown**	`docs/HANDOFF.md`, `CHANGELOG.md`	See section 5

On the two withdrawn figures: I swept ten combinations of budget (12–66), cap (32, 66) and nature (0.9, 1.0, 1.1) over the Regulation M-B roster. The mean HP/(2×Def) ranged 0.83–1.04 and the maximum never exceeded 1.52. A maximum of 5.50 requires a Chansey- or Blissey-class stat line, and neither is in the Champions roster (`CHAMPIONS_IDS_MA.has(113)` and `.has(242)` are both `false`). The figures almost certainly came from a different, larger population — but the population was never recorded, which is precisely the finding. Every figure in that section of `BACKLOG.md` now names its roster, budget, cap and nature.

---

5. Causal claims audit

The brief asked me to check "the store duplication count and its stated cause" against the changelog and `.gitattributes` history. **That claim does not belong to this project.** HoopaDex has no append-only store; the four-way duplication is ABRA's. I checked: HoopaDex has no `.gitattributes` at all, and no store to duplicate. Repeating that finding here would have been the exact error this review exists to catch, so it is recorded as not applicable rather than answered.

The analogous asset in HoopaDex is the live site plus the committed derived data. Its integrity question is F1, and the answer is that nothing guards it.

Claim	Verdict	Evidence
A non-raw Python string turned `\2195` into a control character, which is how the sort arrows became the literal text "95"	**VERIFIED**	`python -c "s='\2195'; print(repr(s))"` → `'\x1195'`. First character U+0011, remainder `95`. The mechanism reproduces exactly.
The three type charts were audited at 838 cells, all correct	**VERIFIED**	$18^2 + 17^2 + 15^2 = 838$ attacker×defender pairs across CM, C2 and C1. I initially mis-read this as declared cells (312) and was wrong; the figure is right and precisely stated.
`PAST_STATS` was 10 of 43 entries correct before 1.96	**DIRECTIONALLY VERIFIED, figures differ**	The pre-fix table at commit `6d3887b` has **42** entries, of which **7** agree with the Showdown derivation, 19 disagree, and 16 describe a revision Showdown records no trace of. The substance — the table was overwhelmingly wrong — holds; the exact count does not.
`ITEM_INTRO_GEN` was 67 of 325 wrong (20.6%)	**UNVERIFIED**	The pre-fix table is not recoverable from this repository's history, which begins after the fix. Cannot be confirmed or refuted.
Speed Tiers was silently missing 148 of 208 Pokémon	**UNVERIFIED**	Same reason. The 208 roster size is confirmed independently.
"Total hits survived scales with HP × (Def + SpD)"	**DISPROVEN as stated**	True for *expected* hits against a wholly-physical-or-wholly-special opponent; false for hits survived against a mixed attacker, where the quantity is HP·Def·SpD/(Def+SpD). Measured divergence: 135 of 208 spreads, worst case 10.49%. See F4.
A point of HP "is worth about twice what a single defence point is" (app UI)	**DISPROVEN**	It is worth (Def+SpD) against one defence point's HP. "About twice" holds only when the defences are close — the folk rule the tool exists to replace. Removed from the UI.
Every suite accepts `HOOPADEX_SRC`	**DISPROVEN**	9 of 23 did not. See F11.
The app is "one HTML file, no dependencies"	**DISPROVEN**	Four files plus a Google Fonts request. See F8.
An auto-commit hook commits and pushes the working tree	**VERIFIED, but not a hook**	`.git/hooks/` is empty and `core.hooksPath` is unset. It is `Projects/auto-publish.bat` on a Startup shortcut. The behaviour is real; the stated mechanism was wrong, and the wrong mechanism sent me looking in the wrong place.

6. What I could not do

- **I never saw the page.** `computer{action:"screenshot"}` failed with "*the Browser pane is not displayed, so the page is not compositing frames*" — the same limitation the previous handoff records. Every visual claim here comes from reading computed values and text out of the DOM. The items previously listed as "measured, never seen" — the light-mode type chart, the hidden-ability

pill, the period-accurate sprites — are **still unseen**, and my changes to the Bulk tab copy join that list. Verified by DOM read, not by eye.

- **I did not change the bulk recommendation.** F4 gives the numbers for both models. Choosing between them is a product decision about how readers actually play, and it changes displayed advice for 135 of 208 Pokémon. Left for Will.
- **I did not pin `@smogon/calc`.** Determining which upstream version the 480 KB bundle came from means diffing it against candidate upstream builds. I could not do that reliably, and guessing a version number would be worse than recording that none is known.
- **I did not fix F9.** Closing it needs the item table regenerated against the live PokéAPI universe, and a network call does not belong in a suite that must pass offline. The measured impact today is zero.
- **I could not verify two historical claims** — the item table's 20.6% error rate and the Speed Tiers gap — because this repository's history begins after both fixes. Marked UNVERIFIED rather than repeated as fact.
- **The white paper and the deck are still stale** (5.3 and 1.3 against an app at 5.9). Bringing them current is a writing task, not an audit task, and doing it badly at the end of a long session is how stale documents get replaced with wrong ones. Recorded as open.
- **I did not disable the auto-publisher** (F1). It is outside this repository and publishes five other projects; stopping it is Will's call.
- **`build/omnibus.py` did not exist**, and the installed `weasyprint` 69.0 cannot load its native libraries on this machine (`WeasyPrint could not import some external libraries`). I wrote the script with a headless-browser fallback so the PDF could still be produced; see section 3.

---

7. The rules

Each rule names the failure that produced it and the check that enforces it. **A rule with no check is a preference, and is labelled as one.** Citations are given where a real body of practice exists, and omitted rather than invented where it does not.

#	Rule	Originating failure	Enforcing check	Real rule?
R1	One repository, one publisher. Publishing runs the tests first and refuses on red.	F1. The timer pushed this review's own work twice. Its own comments record 312 failed pushes from two publishers on ABRA.	**None today.** Requires adding a test gate to `auto-publish.bat`, or removing HoopaDex from it and publishing via `build/publish.sh` as ABRA does.	**Preference until built.**
R2	A value derivable from a published artefact is derived, committed to `data/`, and compared to the app on every run.	F6, F7. Tables with a derivation caught their mutation; tables without one did not.	Drift checks in `test-past-stats`, `test-gen1-special`, `test-generation-tables`, `test-regulation-items`, `test-mega-abilities`, `test-champions-roster`. 6 of 9 data files.	**Rule**
R3	A test that samples a table does not defend it. Assert every entry against a derivation, or state in the file that it is a sample.	F7. 8 of 58 species pinned; the other 50 were free.	`test-past-stats` now compares all 58 values.	**Rule**
R4	A check that compares two artefacts from the same source proves consistency, not correctness — say so where it is used.	F9. 325 === 325 could not see that the universe was 370.	Comment in `test-champions-roster.js` states the limit explicitly. No automated check.	**Preference** — the honesty is enforced by review, not by code.
R5	A new suite must be proven to fail. Mutate the source, confirm red, then commit.	F2. 778 assertions and five deliberate bugs walked through.	**None automated.** The harness exists (mirrored tree + `HOOPADEX_SRC`) and this review used it, but nothing requires it.	**Preference until a mutation run is part of CI.**
R6	Behaviour is tested by computing a value, not by matching source text.	F2. 100 of 473 assertions were regex-against-source; they detect deletion, not wrongness.	Partially: `test-damage-formula` is behavioural throughout. The 100 text assertions remain.	**Preference**
R7	Nothing but the current app is published.	F5. A v1.92 dex was live and wrong.	`test-syntax.js` fails on any page in `app/` other than `index.html`. Verified by mutation M11.	**Rule**
R8	Every published figure names its population and parameters.	Section 4. Two figures unreproducible because the roster was never recorded.	**None automated.** `BACKLOG.md` now states roster, budget, cap and nature for each figure.	**Preference**
R9	An unknown key fails loudly; it does not take a default.	F9. `ITEM_INTRO_GEN[name] || 9` silently assigns a generation.	**None.**	**Preference**
R10	Generation-dependent constants take the	F3. A hardcoded 1.5 crit multiplier in a generation-	`test-damage-formula` pins `calcLocalCritMult` at Gens 3,	**Rule**

#	Rule	Originating failure	Enforcing check	Real rule?
	generation as an argument.	accurate dex.	5, 6 and 9.	
R11	The version on line 2 and the newest CHANGELOG entry must agree.	F10. The external gate is red for unrelated reasons and gets skipped.	`test-syntax.js`, in the battery that must pass.	**Rule**
R12	Before splicing this file programmatically, assert the match count and that the diff is proportionate.	The `\2195` corruption — verified in section 5 — shipped for eight versions.	Partially: `test-syntax.js` scans the whole file for control characters. The match-count discipline is convention; this review's harness enforced it on itself.	**Preference**

Grounding

- **R5 (prove the test fails)** is mutation testing, and the practice has a substantial literature. DeMillo, Lipton and Sayward introduced it in "*Hints on Test Data Selection: Help for the Practicing Programmer*" (IEEE Computer, 1978). Jia and Harman's "*An Analysis and Survey of the Development of Mutation Testing*" (IEEE TSE, 2011) surveys the field. Papadakis et al., "*Are Mutation Scores Correlated with Real Fault Detection?*" (ICSE 2018), is the empirical support for the specific claim this review relies on: mutation score tracks real fault detection where statement coverage does not.
- **R2 (derive, do not transcribe)** is the single-source-of-truth principle; the closest named practice is treating generated artefacts as build outputs rather than source, as in Google's monorepo tooling. I know of no canonical paper and am not going to invent a citation.
- **R6 (behaviour over source text)** corresponds to the standard warning against change-detector tests — tests that fail when code changes rather than when behaviour breaks. Documented in Winters, Manshreck and Wright, *Software Engineering at Google* (O'Reilly, 2020), chapter 12.
- **R9 (fail loudly)** is the fail-fast principle; Nygard, *Release It!* (2nd ed., Pragmatic Bookshelf, 2018) is the usual reference for why silent defaults become invisible faults.
- **R1 (one publisher, gated)** — the specific arrangement here is unusual enough that I would be reaching to attach a citation. The evidence for it is local and sufficient: the script's own header records what two publishers cost.
- **R8 (provenance)** has real literature in reproducible research — Peng, "*Reproducible Research in Computational Science*" (Science, 2011). Whether it transfers cleanly to a hobby dex is arguable, and I mention it as an analogy rather than an authority.
- **R3, R4, R7, R10, R11, R12** are local lessons from this codebase. No literature is claimed.

---

8. What I would do next, in order

1. **Gate the publisher** (R1). Either add `for f in tests/test-*.js; do node "$f" || exit 1; done` before the `git push` in `auto-publish.bat`, or remove HoopaDex from it. This is the highest-value change available and it is a few lines.
2. **Decide the bulk objective** (F4). The numbers are in this document.

3. **Bring the white paper and deck current**, then consider whether `check_projects.py` can compare a version string inside each document to the app.
4. **Add a mutation run to CI** (R5), even a small fixed set. Without it, R5 stays a preference.
5. **Pin** `@smogon/calc` to a known upstream commit and record it.